Studio Nx

Backend(FastAPI/Python) 63 — ProjectSession 나머지 컬럼

2026-10-12

Backend(FastAPI/Python) 63 — `ProjectSession` 나머지 컬럼 (project_sessions)

항목	내용
파일	`studionx/backend/app/models/project_session.py`
작업	P1-16 대시보드 API: 작업 세션 모델 완성 (PRD 01 FR-128, FR-130, DB.md §3.1)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	62 프로젝트 라우터 · 64 `get_storage()`

1. 하는 일

`project_sessions` 표는 시퀀서를 연 기록을 담는 표입니다. 이 표의 ORM 모델에 **아직 없던 칸 3개**를 추가합니다. 표 자체는 이미 DB에 있었고(`studionx/db/01_phase1_account.sql`), 모델 쪽만 템플릿 TODO로 남아 있었습니다.

칸	타입 (DB)	기본값	뜻
`started_at`	`DATETIME(3)` NOT NULL	지금 시각 (DB가 채움)	시퀀서에서 프로젝트를 연 시각 (UTC)
`last_seen_at`	`DATETIME(3)` NOT NULL	지금 시각 (DB가 채움)	마지막 신호 시각: 시퀀서가 1분마다 갱신합니다 (Phase 2)
`active_seconds`	`INT UNSIGNED` NOT NULL	0	5분 넘게 입력이 없던 구간을 뺀 실제 작업 시간 (초)

예를 들어 한국 시각 오후 3시에 시퀀서를 열어 3시 45분까지 작업했고, 중간에 5분 넘게 쉰 적이 없다면 `started_at`은 `06:00:00`(UTC), `last_seen_at`은 `06:45:00`, `active_seconds`는 `2700`이 됩니다.

순서를 이렇게 정한 이유: 다음에 만들 함수들(최근 프로젝트, 작업 시간, Ready to start)이 이 칸들을 읽습니다. 모델에 칸이 없으면 `ProjectSession.started_at` 같은 조건을 코드로 쓸 수 없기 때문에, 모델을 먼저 완성했습니다.

2. 언제 쓰나

쓰는 곳	하는 일
66 `get_recent_projects` (예정)	`started_at`이 최근 7일 안인 내 세션을 읽어 마이페이지 작업 시간 그래프에 씁니다 (FR-128)
67 `get_ready_projects` (예정)	내 세션이 하나도 없는 프로젝트를 찾아 Ready to start 배너에 씁니다 (FR-130)
P1-14 `to_item`, `list_projects`	`opened` 값은 이미 `session_id`, `project_id`, `user_id`만으로 계산하고 있어서 영향이 없습니다
Phase 2 시퀀서 (FR-257)	시퀀서를 열 때 행을 만들고, 1분마다 `last_seen_at`과 `active_seconds`를 갱신합니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime

from sqlalchemy import BigInteger, ForeignKey, text
from sqlalchemy.dialects.mysql import DATETIME, INTEGER
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`datetime`, `text`, `DATETIME`, `INTEGER`를 이번에 추가했습니다. 세 칸의 타입과 DB 기본값을 정하는 데 씁니다.

클래스 (추가한 부분)

```
    # (구현: AI — 2026-10-12, P1-16) 시각은 UTC · 시간대 없이 저장 (다른 시각 칸과 같음)
    started_at: Mapped[datetime] = mapped_column(  # 시퀀서에서 연 시각
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    last_seen_at: Mapped[datetime] = mapped_column(  # 마지막 신호 (1분마다, Phase 2)
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    active_seconds: Mapped[int] = mapped_column(  # 5분 넘게 입력 없는 구간은 뺀 작업 시간
        INTEGER(unsigned=True), server_default=text("0")
    )
```

문법 풀이

import 문법	뜻
`from sqlalchemy.dialects.mysql import DATETIME, INTEGER`	MySQL 전용 타입입니다. 소수 자리(`fsp`)와 부호 없음(`unsigned`)을 정할 수 있습니다
`from sqlalchemy import text`	SQL 문장을 그대로 넘깁니다. 기본값 `CURRENT_TIMESTAMP(3)`을 DB가 채우게 할 때 씁니다

식	뜻	예
`Mapped[datetime]`	파이썬에서 이 칸은 `datetime`이고, `| None`이 없으므로 NULL을 허용하지 않습니다	
`DATETIME(fsp=3)`	소수 셋째 자리(밀리초)까지 저장합니다	`2026-10-10 03:00:00.123`
`INTEGER(unsigned=True)`	0 이상의 정수만 받습니다. 음수를 넣으면 DB 오류가 납니다	`-1` → 오류
`server_default=text("…")`	값을 넣지 않으면 파이썬이 아니라 **DB가** 기본값을 채웁니다	지금 시각, 0

4. 흐름

모델에 칸이 생겼을 뿐, 새로 동작하는 기능은 없습니다. 행을 넣을 때 시각과 작업 시간을 주지 않으면 DB가 지금 시각과 0으로 채우고, 읽을 때는 시간대 정보가 없는 `datetime`(UTC 값)으로 돌아옵니다.

5. 왜 이렇게

선택	이유
SQL 정의와 **똑같이** 맞춤 (`DATETIME(3)`, `INT UNSIGNED`, 기본값)	모델과 DB가 어긋나면 시험과 실제 운영에서 다르게 동작합니다. 그래서 시험에서 칸 이름, 순서, NULL 여부, FK를 실제 DB와 비교합니다
기본값은 DB가 채움 (`server_default`)	`projects.created_at` 같은 다른 시각 칸과 같은 방식입니다. 서버 시계가 아니라 DB 시계 하나를 기준으로 삼습니다
UTC로, 시간대 정보 없이 저장	다른 시각 칸과 같은 규칙입니다. 요일이나 "오늘"을 나누는 일은 화면이 브라우저 시간대로 합니다 (DB.md §3.1)
`active_seconds`를 따로 둠	`last_seen_at - started_at`으로 계산하면 창을 켜 둔 채 쉰 시간까지 포함되어 실제 작업 시간과 달라집니다

6. 시험 (`tests/test_project_session_model.py` 새 파일, 4개)

시험	확인한 것
컬럼 비교	6칸의 이름, 순서, NULL 불가, FK 2개(`project_id`는 ON DELETE CASCADE)가 실제 MySQL과 같습니다
기본값	칸을 비우고 넣으면 두 시각이 시간대 정보 없이 채워지고, `active_seconds`는 0입니다
값 보존	`03:00:00.123`의 밀리초와 2400초가 그대로 저장됩니다
음수	`active_seconds = -1`을 넣으면 DB 오류가 납니다 (UNSIGNED)

전체 시험 496개가 통과했고, `ruff` 검사도 통과했습니다.